

TP3 - Introduccion a Tests Unitarios

Laboratorio de Programación y
Lenguajes

Trabajo Práctico N°3

Introducción a tests unitarios

Fecha de entrega: Martes 7/4 23 hs

Link: <https://classroom.github.com/a/jLxPRyso>

Enunciado

Dominio: Desarrollo de una mini-biblioteca de calculadora y su suite de tests unitarios.

¿Por qué es relevante? En la industria, los tests unitarios permiten detectar errores temprano, documentar el comportamiento esperado y refactorizar con seguridad; pytest es una herramienta estándar para automatizar estas verificaciones en proyectos Python.

¿Qué van a construir?

Un repositorio con:

- un módulo `calculator` que implementa operaciones matemáticas comunes (funciones puras y determinísticas),
- una suite de tests con `pytest` (incluyendo casos borde y manejo de excepciones),
- autograding con GitHub Actions y cobertura mínima.

Comenzar a trabajar

- Loguearse en github
- Acceder al link
- Aceptar la tarea
- Trabajar en el navegador a través de Codespaces (botón superior en el

README.md) o localmente con su IDE preferido

REQUISITOS TÉCNICOS

ESTRUCTURA:

- [] `add(a: float, b: float) -> float`
- [] `sub(a: float, b: float) -> float`
- [] `mul(a: float, b: float) -> float`
- [] `div(a: float, b: float) -> float`

Laboratorio de Programación y Lenguajes

- Debe **lanzar** ``ZeroDivisionError`` si ``b == 0``.
- [] ``pow_(a: float, b: float) -> float`` (usar nombre ``pow_`` para no pisar built-in ``pow``)
- [] ``sqrt(x: float) -> float``
 - Debe lanzar ``ValueError`` si ``x < 0``.
- [] ``mean(values: list[float]) -> float``
 - Debe lanzar ``ValueError`` si la lista está vacía.

Tests:

- [] 1 archivo de tests por grupo (mínimo 4 archivos; recomendado: 1 por tema).
- [] Usar ``pytest`` con:
 - [] asserts claros
 - [] tests de excepciones (``pytest.raises``)
 - [] al menos 1 test parametrizado (``@pytest.mark.parametrize``)
- [] Cobertura mínima: **80%** sobre ``src/``.

Git:

- [] Realizar **6 commits** significativos (uno por ejercicio como mínimo).
- [] No commitear entornos virtuales (``.venv/``) ni caches.

Estructura del repositorio

La entrega debe respetar esta estructura (para que el autograding funcione):

```
.  
??? src/  
?   ??? __init__.py  
?   ??? calculator.py  
??? tests/  
?   ??? __init__.py  
?   ??? test_aritmetica.py  
?   ??? test_division_y_excepciones.py  
?   ??? test_potencias_y_raiz.py  
?   ??? test_estadistica.py  
??? requirements.txt  
??? README.md  
??? .gitignore
```

Entrega final

Se entrega por GitHub Classroom: push a `main` del repositorio que se crea automáticamente al aceptar la tarea antes del deadline.
La corrección automática se ve en GitHub ? Actions.

DOCUMENTACIÓN

Concepto

Python

Recurso

Tipo

<https://docs.python.org/es/3.13/>

Documentación

git

<https://git-scm.com/docs>

Documentación

Machete de git

pytest

<https://education.github.com/git-cheat-sheet-education.pdf>

Guía

<https://docs.pytest.org/en/stable/>

Documentación